SmartRoute AI — Product Requirements Document (PRD)

DOCUMENT: PRD-v1.0
AUDIENCE: Engineers & Judges
STATUS: MVP Completed

Product Specifications, Scoring Engine & Core Features — Explaining How It Works in Simple English

Product Overview & Goal:
SmartRoute AI is an intelligent, traffic-aware routing system composed of an **Express REST API** and an **interactive React/Leaflet dashboard**. Given any origin and destination, it retrieves up to 3 driving alternatives, ranks them mathematically using customizable penalty weights for traffic delay, and provides human-readable reasons for every recommendation.

1. How It Works in Real-Time: Step-by-Step Flow

To easily explain how the system works to judges, here is the exact 5-step journey of a single user request:

Step	System Action	Real-Time Example (San Francisco: Market St to Presidio)
Step 1 Input	**Geocodes Locations**	User enters origin 'Market St, SF' and destination 'Presidio, SF'. Geocoding service converts them to GPS coordinates via TomTom (or instant offline dictionary fallback).
Step 2 Ingest	**Fetches 3 Paths**	Queries TomTom Routing API for up to 3 alternative paths. Ingests distance (km), standard travel time (mins), and live traffic congestion delay (mins).
Step 3 Score	**Applies Scoring Formula**	Applies composite weighting: Score = (50% × Time) + (35% × Traffic Delay) + (15% × Distance). Heavily penalizes routes with stop-and-go gridlock.
Step 4 Classify	**Traffic Severity & Tags**	Assigns traffic category: **Green (Low delay)**, **Yellow (Moderate)**, or **Red (Heavy)**. Generates tags: *'Fastest Route'*, *'Avoids 12m congestion'*.
Step 5 Display	**Interactive Dashboard**	Draws color-coded polyline paths on Leaflet map. Displays route cards and activates a 60-second auto-refresh countdown to monitor traffic shifts.

2. Core Functional Requirements (What We Built)

Feature ID	Module Name	What It Does (Simple English)	Status
FR-1	**Smart Geocoder**	Converts plain text city/street names to coordinates. If internet or API fails, instantly switches to offline dictionary so the demo never crashes.	**Complete**
FR-2	**Multi-Route Fetcher**	Requests candidate driving routes from TomTom Routing API with real-time live traffic delay metrics.	**Complete**
FR-3	**Weighted Scoring Engine**	Ranks routes mathematically based on Time, Delay Penalty, and Distance. The route with the lowest penalty score is recommended as #1.	**Complete**
FR-4	**Severity Engine**	Classifies traffic delay: Green (<3 mins), Yellow (3–8 mins), Red (>8 mins). Colors map polylines accordingly.	**Complete**
FR-5	**API Key Security**	Requires X-API-Key header. Keys are cryptographically hashed using SHA-256. Includes rate-limiting (100 req/min).	**Complete**
FR-6	**Leaflet Map Dashboard**	Interactive React 18 map showing colored polylines, start/end pins, route summary cards, and quick city preset buttons (SF, NYC, London).	**Complete**
FR-7	**Live Auto-Refresh**	A visual 60-second circular countdown timer that automatically refreshes delay metrics without reloading the page.	**Complete**

SMARTROUTE AI — PRODUCT REQUIREMENTS DOCUMENT (PAGE 2)

3. Technical Architecture, APIs, NFRs & Roadmap

3.1 System Architecture: Clean 3-Tier Design

The application is built on a modular, decoupled architecture: **Frontend:** React 18 + Vite + Leaflet maps + Bootstrap 5. **Backend:** Node.js + Express.js REST API with input validation & SHA-256 authentication. **Traffic Layer:** TomTom Routing & Traffic API with an intelligent local offline fallback dictionary.

3.2 REST API Endpoints Specification

Endpoint	Method & Auth	What It Does & Expected Response	Response Time
/api/routes/recommend	**POST** X-API-Key required	Accepts origin & destination (text or coords). Returns ranked routes with GeoJSON coordinates, duration, traffic delay, and tags.	< 350ms (Live) < 30ms (Offline)
/api/routes/refresh	**POST** X-API-Key required	Refreshes live traffic delay and re-computes ranking scores for existing routes without re-geocoding locations.	< 250ms (Live) < 20ms (Offline)
/api/health	**GET** Public (No auth)	Returns system status, TomTom connectivity state, and server uptime.	< 15ms
/api/developer/status	**GET** X-API-Key required	Validates API key and returns remaining quota and rate-limit status.	< 20ms

3.3 Performance, Security & Test Coverage

Metric / Requirement	Target Specification	How We Achieved It
API Speed (Latency)	< 450 ms response time	Streamlined native fetch calls, light JSON payloads, and local coordinate lookup.
Zero-Crash Reliability	100% demo availability	Built-in fallback dictionary ensures routes display even if internet or TomTom quota fails.
API Security	SHA-256 hashed keys	Keys are never stored in plain text; rate-limiter prevents spam or denial of service.
Automated Test Suite	22 unit & integration tests	100% passing tests covering auth, ranking algorithm, severity, and health endpoints.

3.4 Product Roadmap (Now vs Next)

Phase	Timeline	Key Features & Deliverables
Phase 1: Hackathon MVP	**Current (Q4 2026)**	Core 3-route ranking engine, Leaflet interactive map, Green/Yellow/Red severity, offline fallback mode, 22 automated tests.
Phase 2: Multi-Stop Fleet	**Q1 2027**	Multi-stop traveling salesperson delivery sequencing, automated webhook alerts on unexpected congestion, Python/Go SDKs.
Phase 3: Smart EV Fleet	**Q2 2027**	Electric Vehicle (EV) battery consumption optimization, hill-incline factor, weather condition overlays, enterprise portal.

3.5 2-Minute Live Demo Script for Judges

Step 1 (Open Dashboard): Show the clean dark-mode Leaflet map and system health badge.
Step 2 (Click Preset): Click 'San Francisco' preset. Watch the origin and destination populate automatically.
Step 3 (Explain the Ranking): Point out how Route #1 is recommended not because it's the shortest distance, but because it bypasses an 18-minute traffic jam. Point to the green delay tag and the mathematical rationale.
Step 4 (Show Real-Time Refresh & Offline Fail-Safe): Point to the 60-second auto-refresh countdown timer and mention the fail-safe offline dictionary that guarantees 100% demo uptime.